

SentinelCut — MVP Scope Checklist (v1)

This trims the full SRS down to what your 4-person team actually builds for the demo, while keeping the profanity detection strong across English, Hindi, and Hinglish — including slang, regex-style variants, and stretched-out spellings (`fuuuuck`, `f*ck`, `f.u.c.k`, `m@darchod`).

Your full documentation package stays as-is for submission (PRD, SRS, architecture, etc.). This file is the **build filter** your team codes against.

1. Cut from MVP (build later / mention only in "Future Scope")

- Admin Panel (user management, DB management, logs, feedback dashboard)
- Analytics / toxicity scoring / emotion detection
- Role-Based Access Control (RBAC) — single user role only, no admin/moderator tiers
- Google OAuth login (email+password only for now)
- Guest mode
- AI voice cloning / custom spoken-word replacement
- Custom uploaded sound replacement (beep + mute only for MVP)
- Real-time/live features (mic censorship, streaming, Zoom/Discord/OBS/browser extension)
- Multi-language subtitle translation, speaker diarization
- Storage usage widget, credits system, processing-queue visualization (keep a simple "processing / done / failed" status only)
- Context-aware transformer/LLM classification (IndicBERT/XLM-R) — **deferred**, not dropped. See §3.4.

2. MVP Core (Must-Have)

- ☐ Upload audio/video (MP4, MP3, WAV to start — add MOV/MKV/AAC/FLAC only if time allows)
- ☐ Speech-to-text via Faster-Whisper with word-level timestamps
- ☐ Language handling: English, Hindi, Hinglish, mixed/code-switched sentences (see §3)
- ☐ Profanity detection: dictionary + regex + normalization (see §3)
- ☐ Custom user word list, merged into detection
- ☐ Replacement: Beep, Mute
- ☐ Transcript/timeline view: see detections, timestamps, accept/remove a detection
- ☐ FFmpeg re-mux preserving sync, resolution, frame rate
- ☐ Export: censored video, censored audio, transcript (.txt), detection metadata (.json)

- ☐ Auth: signup/login/logout (email+password), basic session
- ☐ Dashboard: recent files + simple status only

3. Profanity Detection Pipeline (this is your differentiator — keep it solid)

Since the input is mixed-language, **don't gate detection by detected language**. Always run the combined multilingual word list against the text regardless of which language Whisper thinks it heard — language detection is used to pick Whisper's decoding hint for *better transcription accuracy*, not to decide which profanity list applies. This is what makes "Ye bahut bakchodi kar raha hai" and "This movie is fucking amazing" both catchable by one pipeline.

3.1 Normalization stage (run before matching)

Every transcribed word passes through this before dictionary lookup:

1. **Lowercase everything.**
2. **Collapse repeated characters** — `fuuuuck` → `fuck`, `bcccc` → `bc`.

python

```
import re
def collapse_repeats(word: str) -> str:
    return re.sub(r'(\w+)\1{2,}', r'\1', word)
```

3. **Leetspeak / symbol substitution mapping** — `@`→a, `3`→e, `1`/`!`→i, `0`→o, `$`→s, `+`→t.

python

```
LEET_MAP = {'@': 'a', '3': 'e', '1': 'i', '!': 'i', '0': 'o', '$': 's', '+': 't'}
def delete_leet(word: str) -> str:
    return ''.join(LEET_MAP.get(ch, ch) for ch in word)
```

4. **Strip separator tricks** — `f.u.c.k`, `f_u_c_k`, `f u c k` → `fuck` (remove non-alphanumeric between single letters, then re-check as one token or as a merged n-gram of adjacent short tokens).
5. **Transliteration normalization for Hinglish** — normalize common spelling variants of the same Hindi word (e.g., `bhenchod` / `behnchod` / `bhenchodh`) to one canonical root using a small variant-mapping table, not a full transliteration model (keeps it buildable in a mini-project timeline).

3.2 Dictionary matching

Maintain word lists as structured JSON, not a flat text file, so severity and language are queryable:

json

```
{
  "word": "bkl",
  "canonical": "behen ki lund",
  "language": "hinglish_abbrev",
  "severity": "high",
  "variants": ["bkl", "b.k.l", "bkl."]
}
```

Categories to cover:

- English profanity (standard open-source list, e.g. a curated `(en)` bad-words set)
- Hindi (Devanagari script) profanity
- Hinglish / romanized Hindi profanity (`(bakchodi)`, `(chutiya)`, `(gandu)`, etc.)
- Common abbreviations/slang (`(bc)`, `(bkl)`, `(mc)`, `(mc)`, `(lol)`-adjacent insult acronyms)
- Custom words added by the user at upload time (merged into the same lookup at runtime, not a separate pass)

3.3 Regex + fuzzy matching (catches spelling mistakes and near-misses)

- Per-root regex patterns to catch known variant spellings (`(fu+ck)`, `(f\W*u\W*c\W*k)`).
- Optional fuzzy matching with `(rapidfuzz)` (Levenshtein distance $\leq 1-2$) against the canonical word list to catch typos the regex misses — cheap to add, meaningfully raises recall.

3.4 Context-aware scoring — Should-Have, not Must-Have

A transformer-based context model (IndicBERT/XLM-R style) helps disambiguate genuinely ambiguous cases, but a well-built normalization + dictionary + regex + fuzzy pipeline realistically catches the large majority of real-world profanity for a demo, with much less implementation and GPU cost. Build the pipeline above first and get it fully working end-to-end; add context scoring afterward only if time remains, as a confidence-boosting layer rather than the primary detector. This also matches your own SRS's Must/Should split (SRS-FR-PDE items are Must; context-model items are Should/Could).

3.5 Output shape per detection (contract between AI pipeline and backend)

json

```
{
  "word": "fuuuuck",
  "normalized": "fuck",
  "language": "english",
  "severity": "high",
  "start": 12.34,
  "end": 12.81,
  "source": "dictionary",
  "confidence": 0.95
}
```

Keep this JSON schema locked early — it's what the frontend timeline editor, the audio-replacement step, and the exported metadata file all consume.

4. Suggested ownership (unchanged from before, detection work sits with Person 1)

Person	Owns	Note for this update
1 - AI/ML pipeline	Whisper + language handling + full detection pipeline (§3)	This is now the heaviest, most important slice — budget extra time here
2 - Backend/API	FastAPI, Celery/Redis queue, Postgres schema	Consumes the JSON shape in §3.5
3 - Audio/Video processing	FFmpeg/PyDub beep+mute, sync, export	Beep/mute only for MVP
4 - Frontend	Upload, dashboard, transcript/timeline viewer	Simple accept/remove per detection, no full re-editing UI needed for MVP

5. Definition of done for the MVP demo

A single MP4 containing an English sentence, a Hindi sentence, and a Hinglish/mixed sentence — each with at least one profane word, including one stretched-out spelling ((fuuuuck)) and one abbreviation ((bc)) — goes in, and comes out beeped/muted at the correct timestamps with a matching transcript and JSON metadata file. If that works reliably, you have a defensible, demoable core product.